RetailPulse –

AI-Powered Customer Analytics & Demand Forecasting Platform

Subtitle: End-to-end data science and analytics solution for retail demand prediction and customer insights

Author: Kritika Mohan

Role: 3-Month Intern

Prepared for: Zidio Development, Data Science & Analytics Domain

Date: November 2026

Project Duration: 1 Month

Live demo: https://retailpulse-kritikamohan.streamlit.app/

Code: https://github.com/kritikaamohan/retailpulse

1. Project Overview

Retailers lose sales when a product runs out and tie up money when they hold too much stock. RetailPulse is an end-to-end data science platform for a retail business. It takes raw sales transactions and produces four decisions: how much will sell (demand forecasting), who the customers are (segmentation), who may stop buying (churn prediction), and what to reorder (inventory optimization). The results are shown in a public Streamlit dashboard, and the models are tracked, tested and monitored with MLOps tools.

The project uses the Online Retail II dataset (UCI), the transactions of a UK online retailer from 1 December 2009 to 9 December 2011. The raw file has 1,067,371 rows and 8 columns: invoice, stock code, description, quantity, invoice date, price, customer ID and country.

1.1. Vision and Objectives

The vision and objectives of this project are:

- Forecast daily revenue for the next 30 days.
- Group customers into meaningful segments with suggested actions.
- Identify customers likely to churn and explain why.
- Turn the forecast into reorder recommendations.
- Track experiments, validate data, and detect data drift.

1.2. Target Users and Use Cases

User	Use case in RetailPulse
Inventory planner	Reads the 30-day demand forecast and the reorder table (reorder point, safety stock, order quantity).
Marketing & CRM team	Uses the six customer segments and the ranked list of at-risk customers for retention campaigns.
Business manager	Tracks revenue, orders and average order value with date and country filters.
Data team	Compares model runs in MLflow, reads drift reports, and checks the retraining trigger.

1.3. Business Value Delivered

- **Revenue is concentrated:** The 472 Champions (8.1% of 5,852 identified customers) account for 54.4% of the revenue from identified customers.
- **Many customers are slipping away:** 47.5% of customers fall in the At-Risk or Lost segments, which gives a clear win-back target.
- **Reorder logic reduces lost sales in simulation:** On simulated stock levels, the optimized policy cut unmet demand by 46% to 78% across three random seeds, at the price of roughly double the average inventory.
- **The forecast follows the weekly cycle:** The shop does not trade on Saturdays, and the final model reproduces this.

1.4. Non-functional goals: targets and results

Goal	Target	Result
Forecast accuracy	MAPE ≤ 12%	19.34% on the 30-day test window; about 14% over the first 7 to 14 days
Churn discrimination	AUC-ROC ≥ 0.88	0.714 (random split), 0.768 (out-of-time)
Churn precision	Precision@top 20% ≥ 0.75	0.658 (random split), 0.686 (out-of-time)
Observability	MLflow tracking and drift detection	Forecast and churn runs logged in MLflow; Evidently drift reports and a retraining trigger
Overstock / understock	Reduce by 25-40%	Unmet demand down 46-78%, inventory up 120-128% (simulated stock)
Load time	< 8 s	Median 4.3 s over 6 reloads. Four loads took 3.0 to 4.5 s, and two took about 17 and 19 s
Daily batch	< 5 min	Data refresh 6.5 s + drift check 40.7 s = about 47 s (about 76 s with the retraining run)
Scale	10M+ transactions per month	One step (the dashboard aggregation) handled 10.0M synthetic rows in 1.5 s; the full pipeline was not tested at this size.

2. Architecture Overview

Raw transactions are cleaned and validated, turned into features, and passed to four independent models. Their outputs are saved as small CSV files in dashboard_data/, and the Streamlit app only reads those files. The app therefore never trains a model, opens quickly and needs no secrets or paid services.

The MLOps layer sits beside the pipeline. MLflow records every model run. Evidently compares new customer features with the training features. If more than 30% of the features drift, src/retrain.py retrains the churn model and logs the run. GitHub Actions runs linting, tests and a Docker build on every push. The Airflow DAG and Kubernetes manifests describe how this would be scheduled and deployed; they were written but not run.

1. Raw sales data: The Online Retail II dataset: about 1.07 million transactions (invoice, product, quantity, date, price, customer, country) from a UK online retailer, December 2009 to December 2011.

2. Data cleaning and validation: Duplicates, cancelled invoices, zero-price rows and non-product codes such as postage are removed, with a count at each step. Rows without a customer ID are kept for forecasting and excluded from customer models. Great Expectations checks the result (no null identifiers, positive price, quantity and revenue).

3. Feature engineering: RFM features per customer, a daily revenue series with rolling averages, and eleven behaviour features for churn (such as the average gap between orders and how overdue a customer is), all built only from data before a cutoff date.

4. Modeling: Four independent models:

- Customer segmentation: K-Means on log-scaled RFM features (six segments), with DBSCAN used to find outliers
- Demand forecasting: Prophet (tuned) with a Saturday closure rule, compared with a seasonal baseline, an LSTM and an ensemble
- Churn prediction: XGBoost with SHAP explanations, tuned with Optuna
- Inventory optimization: reorder points, safety stock and order quantities calculated from the forecast

5. Saved results: Each model writes its outputs to small CSV files in dashboard_data/, so the dashboard shows precomputed results.

6. Streamlit dashboard: Five pages: Overview (KPIs, alerts, filters), Demand Forecast (model comparison and a what-if slider), Segments and Churn, Inventory, and Monitoring. It is hosted on Streamlit Community Cloud, and every table can be downloaded as CSV.

7. MLOps layer:

- *MLflow* records the parameters, metrics and predictions of every forecasting and churn run.
- *Evidently* compares new customer data with the training data and produces drift reports.
- *Retraining script* (retrain.py): if more than 30% of features drift, the churn model is retrained and the run is logged.
- *GitHub Actions* runs linting, 12 tests and a Docker build on every push.
- *Airflow DAG and Kubernetes manifests* describe a weekly schedule and a cluster deployment. They are written but were not run.

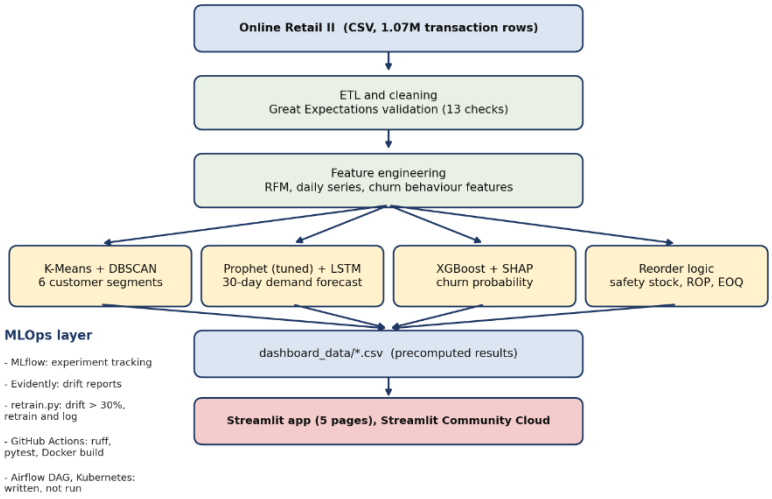


Figure 1. High-level data flow and MLOps layer of RetailPulse.

3. Execution Timeline

The work followed the 28-day plan in the brief, organized by week. It was carried out in October 2026.

Phase	Brief days	Work completed	Milestone
Week 1: data and baselines	1-7	EDA notebook; cleaning with row counts; RFM and rolling features; Great Expectations; K-Means and DBSCAN; time-series preparation, ADF test and decomposition; Prophet and LSTM baselines; MLflow logging.	Cleaned dataset, EDA, baselines logged in MLflow.
Week 2: modeling and MLOps	8-14	Prophet + LSTM ensemble; XGBoost churn model with SHAP; inventory logic; Optuna tuning; Evidently drift detection; retraining script and Airflow DAG file.	Forecasting, churn and inventory logic ready.
Week 3: dashboard	15-21	Streamlit pages for overview, forecast with what-if slider, segments and churn, inventory and monitoring; CSV downloads.	Interactive dashboard with all insights.
Week 4: deployment	22-28	Deployment on Streamlit Community Cloud; multi-stage Dockerfile; Kubernetes manifests; GitHub Actions CI; 12 tests; forecast tuning; report, README and demo video.	Live public URL and green CI run.

4. Technical Highlights

4.1. Data Exploration and Cleaning

Exploration found that the raw data mixes real sales with cancellations, stock adjustments and fees. For example, quantity ranges from -80,995 to +80,995 (one large order and its cancellation), price has a minimum of -53,594, and 22.77% of rows (243,007) have no customer ID. The 3,457 negative-quantity rows that are not cancellations have price 0, no customer and staff notes such as "lost" or "damages", so they are warehouse corrections and not demand.

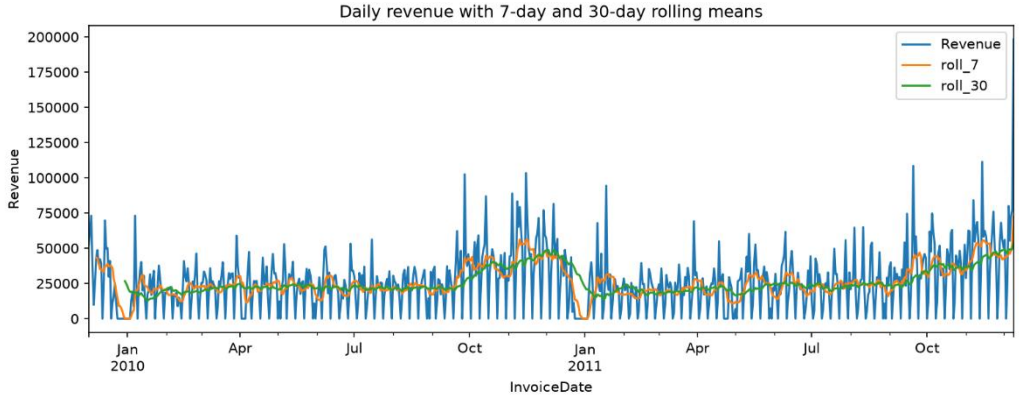


Figure 2. Daily revenue with 7-day and 30-day rolling means.

Cleaning step	Rows removed	Reason
Exact duplicate rows	34,335	Identical lines within the same invoice are most likely system duplicates.
Cancelled invoices (start with C)	19,104	Returns or voided orders are not sales.
Price ≤ 0	6,019	Adjustments and samples; also removes the stock-adjustment rows.
Quantity ≤ 0	0	Safety check; the earlier rules had already removed these rows.
Non-product stock codes (POST, DOT, M, ...)	4,699	Fees and adjustments. The rule also removed a few gift-voucher and DCGS codes.
Rows kept for sales analysis	**1,003,214**	94% of the 1,067,371 raw rows.
Rows with a customer ID (customer models)	776,577	5,852 customers. Rows without an ID are kept for demand forecasting only.

Great Expectations validated the cleaned sales, customer and RFM tables with 13 checks (no null identifiers, positive price, quantity and revenue, unique customer IDs), and all of them pass.

4.2. Feature Engineering

- **RFM:** Recency (days since the last purchase, from 10 Dec 2011), Frequency (number of unique invoices) and Monetary (total revenue) for each customer. The three features are heavily right-skewed, so they are log-transformed and standardized before clustering.
- **Daily Series:** Total revenue per day, with 7-day and 30-day rolling means.
- **Churn Features** (11, all computed only from data before the cutoff): Recency, Frequency, Monetary, number of distinct products, tenure, average order value, average gap between orders, overdue ratio (recency divided by the customer's usual gap), orders in the last 30 and 90 days, and the share of orders that are recent.

4.3. Customer segmentation

K-Means was run for k = 2 to 10. Silhouette scores for the allowed range were 0.335 (k = 6), 0.303 (k = 7) and 0.295 (k = 8), so k = 6 was chosen. DBSCAN found only 2 to 3 clusters (eps 0.3: 3 clusters, 370 noise points, silhouette 0.165; eps 0.4: 2 clusters, 149 noise points, silhouette 0.280; eps 0.5: 2 clusters, 83 noise points, silhouette 0.278), because RFM data forms one continuous cloud. DBSCAN was therefore used only to flag outlier customers.

Segment	Customers	Share	Median recency (days)	Median frequency	Median spend (£)	Suggested action
Champions	472	8.1%	8	22	9,227	Reward and protect
Loyal Customers	912	15.6%	60	9	3,350	Upsell, loyalty program
Potential Loyalists	830	14.2%	14	5	1,396	Encourage repeat purchase
Recent Low-Spend	859	14.7%	41	2	387	Welcome series, raise basket size
At-Risk	1,308	22.4%	281	3	984	Win-back offer
Lost / Hibernating	1,471	25.1%	425	1	227	Low-cost reactivation

A silhouette score of 0.335 indicates usable but not sharply separated groups, which is typical for customer data. The segments cover identified customers only.

4.4.Demand Forecasting

The daily revenue series runs from 1 December 2009 to 8 December 2011. The last, partial day (9 December 2011) was excluded. The series has 135 zero-revenue days, mostly Saturdays (average revenue on Saturdays is about 93, against 37,830 on Thursdays) plus the Christmas closure. The Augmented Dickey-Fuller test gives $p = 0.158$ for the original series (non-stationary) and $p < 0.0001$ after one difference (stationary). Decomposition shows a trend that rises into the autumn peak in both years, a strong 7-day cycle, and residuals that widen in the busy months.

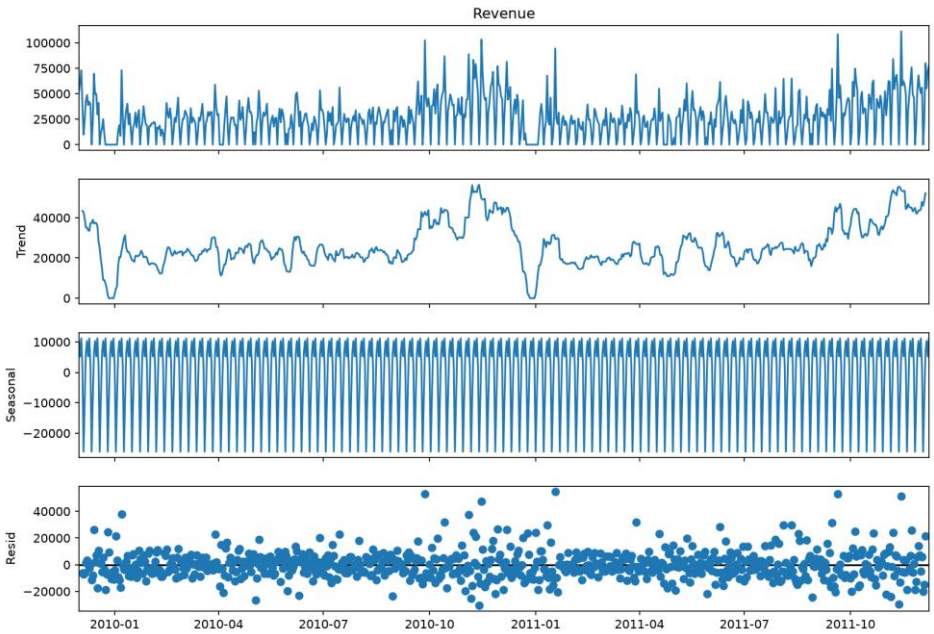


Figure 3. Decomposition of daily revenue (period 7): observed, trend, weekly seasonal and residual.

The models were trained on data up to 8 November 2011 (708 days) and tested on the following 30 days. Two metrics are reported. **MAPE** is computed only on trading days, because on a Saturday the

actual value is zero and the percentage error is undefined. **WAPE** (total absolute error divided by total actual revenue) covers all days.

Model	MAPE, trading days	WAPE
Seasonal naive	29.37%	25.53%
Prophet baseline	20.83%	26.06%
LSTM	33.32%	36.31%
Ensemble, 50/50 Prophet and LSTM	21.40%	27.48%
Prophet + Saturday closure rule	20.83%	19.80%
Prophet tuned + Saturday rule (final)	**19.34%**	**19.49%**

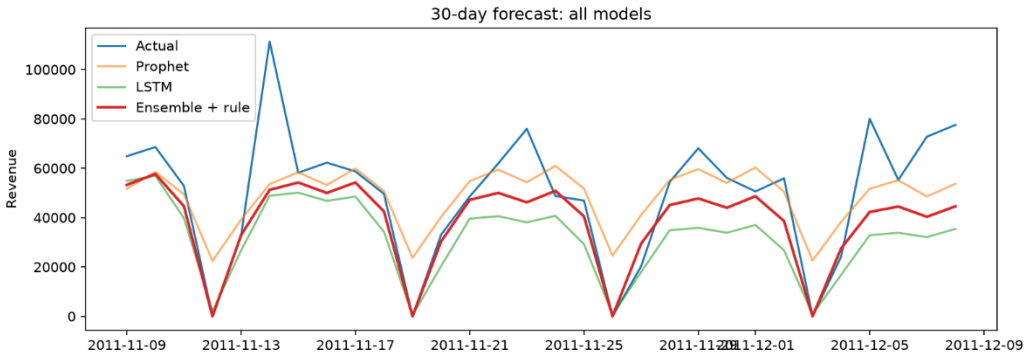


Figure 4. Actual revenue and forecasts from Prophet, the LSTM and the ensemble with the Saturday rule (30-day test window).

The LSTM learned the Saturday closure but under-predicted the level of sales on trading days, so averaging it with Prophet pulled the forecast down and the ensemble did not beat Prophet. The best earlier model was Prophet with a rule that sets Saturday to zero, which is learned from the training data. Five Prophet configurations were compared on three earlier 30-day windows from the training data, and the test window was used once. The differences are small, so the improvement from tuning is modest.

Configuration	Validation MAPE	Validation WAPE
Log-transform + UK holidays (selected)	31.79%	29.53%
Current: zeros kept, additive	32.27%	29.13%
Closed days treated as missing	33.59%	29.26%
+ multiplicative seasonality	34.80%	29.78%
+ UK holidays (multiplicative)	34.86%	29.81%
Log-transform + UK holidays (selected)	31.79%	29.53%

MAPE is computed on trading days because the shop is closed on Saturdays and the actual value is zero. The three earlier validation windows average about 32% MAPE, so the test window is the easier one. Weekly (7-day total) MAPE is 9.9% on the test window and 17.5% on average over the validation windows, which is a rough supplementary view.

Horizon	MAPE (trading days)	WAPE
7 days	14.30%	18.76%
14 days	14.21%	16.76%
21 days	18.91%	18.80%
30 days	19.34%	19.49%

4.5.Churn prediction

Churn is defined as no purchase in the 90 days after a cutoff date of 9 September 2011, among customers who were active in the 180 days before it. Features use only data before the cutoff and the label uses only the following 90 days, which avoids data leakage. The sample has 2,775 customers, of whom 1,068 (38.5%) churned. A stratified 80/20 split gives a test set of about 555 customers. SHAP shows that high spend, many different products and recent orders lower the risk, while long gaps between orders and being overdue raise it.

Model	CV AUC	Test AUC (random split)	Precision@top 20%
Recency-only rule	-	0.616	-
Logistic regression	-	0.706	-
XGBoost, default parameters	0.769	0.699	0.604
XGBoost, tuned with Optuna	0.780	0.714	0.658

The tuned model beats the recency-only rule clearly and the logistic regression slightly. Its precision of 0.658 for the top 20% of customers is about 1.7 times the 38.5% base rate. It is below the 0.88 AUC and 0.75 precision targets. The test set is small, so differences of about 0.02 AUC are within noise, and the cross-validated 0.78 is the steadier estimate. The data limits the result: the history is two years, and customer behaviour before the cutoff carries limited information about the next 90 days.

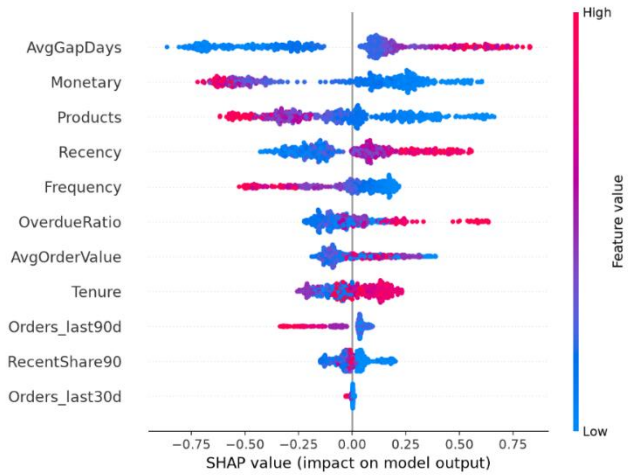


Figure 5. SHAP summary for the churn model. Each dot is a customer; red means a high feature value.

Out-of-time model (tested on 9 Sep 2011)	AUC	Precision@top 20%
XGBoost, trained on 12 earlier cutoffs	0.768	0.686
Logistic regression, same setup	0.749	-
Recency-only rule	0.658	-

To test the model the way it would be used, I also trained it on 12 earlier cutoff dates (every 30 days from 9 July 2010) and tested it on the latest one, 9 September 2011, with 2,775 customers. XGBoost reached an AUC of 0.768 and precision@top 20% of 0.686, about 1.8 times the base rate, which agrees with the cross-validated 0.78. Three caveats: the churn rate differs between training (52.6%) and test (38.5%), so the dashboard keeps the original model; only one future cutoff was tested; and the tuned parameters were selected on data from the same cutoff.

4.6.Inventory Optimization

The dataset has no stock levels or supplier lead times, so these were **simulated**: lead time is 5 to 14 days and starting stock is 0.3 to 3 times the lead-time demand, both drawn at random. Demand, the forecast growth factor (1.176, from the earlier Prophet forecast) and the formulas are real. For the 30 top products by recent revenue:

- **Safety stock** = 1.65 × demand standard deviation × square root of lead time (95% service level).
- **Reorder point** = average daily demand × lead time + safety stock.
- **EOQ** = square root of (2 × annual demand × order cost / (holding rate × unit price)), with an assumed £50 order cost and 20% holding rate.

A 30-day simulation on actual demand (9 November to 8 December 2011) compared a naive policy (no safety stock, 14 days of demand per order) with the optimized policy, using the same simulated stock. The margin lost on an unmet sale was assumed to be 40%.

Random seed	Unmet demand	Average inventory	Total cost
42	-54.1%	+120.3%	-52.5%
1	-46.0%	+127.5%	-41.2%
2	-78.3%	+123.2%	-68.5%

The direction is the same every time: fewer lost sales, higher inventory and lower total cost under these assumptions. With seed 42, unmet demand fell from 28.1% to 12.9%. These figures demonstrate the method and are not predicted savings for a real retailer. The inventory logic used the earlier forecast and was not recomputed after the tuning step.

4.7.MLOps practices

- **MLflow:** Every forecasting and churn model run is logged with parameters, metrics and prediction files.
- **Validation:** Great Expectations checks guard the cleaned data; pytest tests the metric functions and the saved results.

- **Drift Detection:** 9% of features drifted on the real test batch, 55% on a deliberately shifted batch, and the mean predicted churn probability rose from 0.395 to 0.482.
- **Retraining trigger:** src/retrain.py validates the data, checks drift, and retrains and logs the churn model if more than 30% of features drift. With the final setup it reported 29% on real data (no retraining) and 71% with simulated drift (retraining).

4.8. Challenges and Its Solution

Challenges	Solution
Zero-revenue Saturdays break MAPE	MAPE on trading days plus WAPE; a closure rule learned from training data.
Data leakage in Churn	Features before a cutoff date, label from the next 90 days
Extreme outliers and Non-sales rows	Investigated first, then removed with a count at each step
Ensemble worse than Prophet	The LSTM under-predicted the level; Prophet with the closure rule was kept
False drift alarms	Same-season reference and features that do not grow with history length
No inventory data	Simulated stock and lead times, clearly labelled in the app and this report.

5. Deployment and Operations

The dashboard is deployed on Streamlit Community Cloud at https://retailpulse-kritikamohan.streamlit.app/. It has five pages: Overview (KPIs, filters, trends, top countries and products), Demand Forecast (model comparison, what-if slider), Segments and Churn, Inventory, and Monitoring (Evidently reports and a drift simulator). Every table has a CSV download. The app loads precomputed files from dashboard_data/, so it needs no login, secrets or paid services.

The live app runs on Streamlit Community Cloud from requirements.txt and app/main.py; it does not use the Docker image or Kubernetes. The multi-stage Dockerfile is built in GitHub Actions on every push. The Kubernetes manifests and the Airflow DAG are written but were not run.

Deployment platform

- Streamlit Community Cloud hosts the dashboard (public URL, no login)
- GitHub for version control, with semantic commit messages
- GitHub Actions on every push: ruff linting, 12 tests, Docker image build (green)

Monitoring setup

- Evidently drift reports on the Monitoring page, with a drift simulator
- Alerts panel on the Overview page: at-risk customers, products below reorder point, drift status
- MLflow experiment tracking

Limitations:

- Forecast MAPE (19.34% on test, about 32% on earlier windows) and churn AUC (0.714) are below the targets in the brief.
- Inventory levels and lead times are simulated, and the inventory logic used the earlier forecast.
- The what-if slider scales forecast revenue and is a scenario, not a trained promotion model.
- Segments cover identified customers only; about 23% of rows have no customer ID.
- The data has no cost information, so only revenue is analyzed, not profit.
- The full pipeline was not tested at 10 million rows; only the dashboard aggregation step was (1.5 s).

6. Visuals

The architecture diagram (Figure 1) and the analysis charts (Figures 2 to 5) appear in sections 2 and 4 respectively. The figures below are captures of the dashboard.

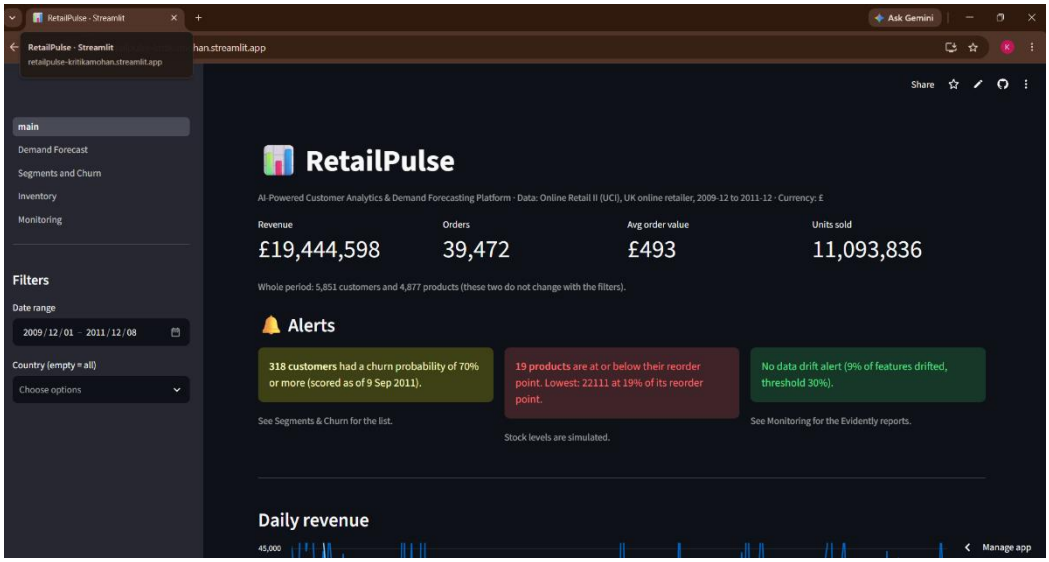


Figure 6. Live Overview page: KPIs, filters and the daily revenue chart.



Figure 7. Demand Forecast page with the horizon slider at 14 days MAPE and WAPE cards, horizon and promo-lift sliders.

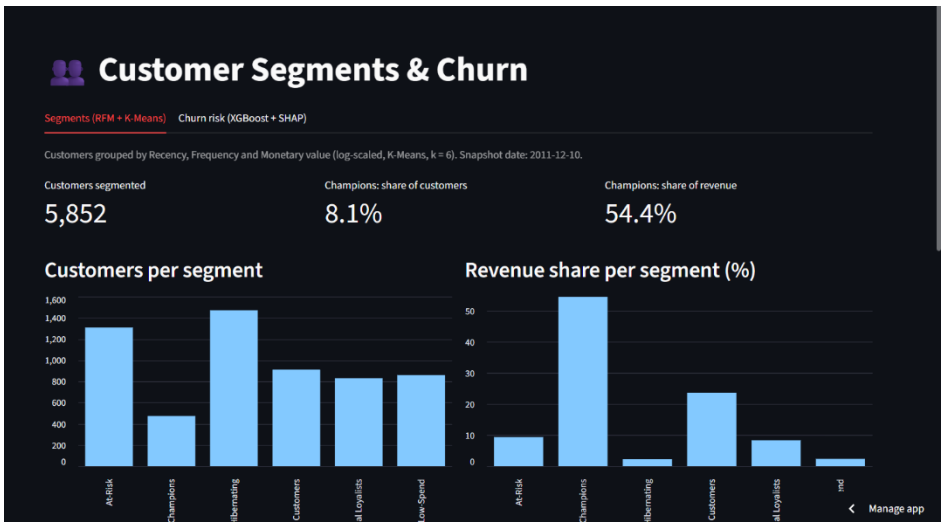


Figure 8. Segments tab: customers and revenue share per segment (Champions: 8.1% of customers, 54.4% of revenue)



Figure 9. Churn tab: model metrics with targets marked as not met, and the at-risk customer list.

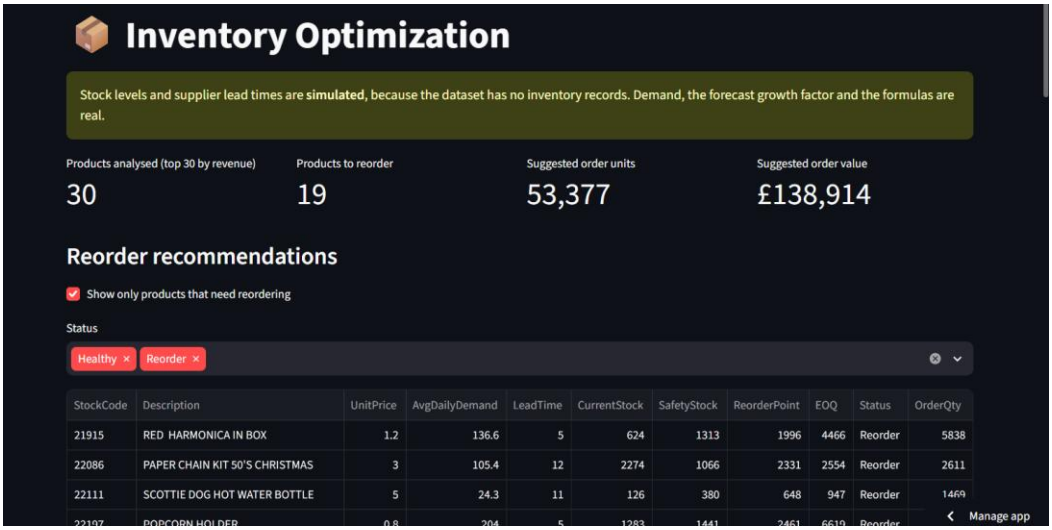


Figure 10. Inventory page: warning that stock is simulated, KPIs and the reorder table.

7. Personal Reflection

Key Learnings

- A metric can be undefined for the data: MAPE breaks on zero-revenue days, and the fix came from understanding the business.
- Data leakage is easy to create; separating features from labels with a cutoff date is the main safeguard.
- A more complex model is not automatically better: Prophet with a simple rule beat the LSTM and the ensemble.
- A monitoring rule needs testing on normal data as well as simulated drift.
- The dashboard, notebooks and documents must come from the same saved results.

Industry best practices applied

- Time-based splits and validation windows for forecasting; stratified split and cross-validation for churn.
- Experiment tracking, automated tests, linting and CI on every push.
- Reproducible environment and clear labelling of assumptions and simulated data.

Future Roadmap

- Add real inventory, cost and promotion data, and product categories, so the inventory and what-if pages use measured inputs.
- Forecast per product, and test weekly forecasts for replenishment decisions.
- Run the Airflow DAG and Kubernetes deployment, and add Prometheus and Grafana monitoring.

Personal growth

Before this project, I thought most of data science was choosing the right model. Most of my time actually went into the data and into checking my own results. The Saturdays with zero revenue broke my MAPE, and I had to understand why before I could trust any forecast number. My first drift checks raised alarms on normal data because I compared different seasons, so I rebuilt it. Later I noticed the dashboard was showing an older forecast than my notebook, and that taught me to rebuild and test the saved results every time. I was surprised that Prophet with one simple Saturday rule beat my LSTM and my ensemble. My forecast and churn scores are still below the targets in the brief, but I know the likely reasons, and I would rather report real numbers than adjust things until they look good. Next time I would plan the validation and the evaluation before building models, and ask for real stock data from the start.

8. Key Features

ID	Feature	Description	Acceptance criteria and result
F-01	Data ingestion and cleaning	Cleaning rules with row counts at each step, plus Great Expectations validation.	Quality checks: 13 checks across three datasets, all passing. Scheduling exists only as an Airflow DAG that was not run.
F-02	Customer segmentation	RFM features, log scaling, K-Means (k = 6); DBSCAN used for outliers.	6 to 8 segments with business meaning: 6 named segments (met).
F-03	Demand forecasting	Prophet (tuned) with a Saturday closure rule, compared with seasonal naive, LSTM and an ensemble.	MAPE ≤ 12%, 30-day horizon: 19.34% (not met).
F-04	Churn prediction	XGBoost with SHAP explanations, leak-free 90-day label.	AUC ≥ 0.88 and precision@top 20% ≥ 0.75: 0.714 and 0.658 (not met).
F-05	Inventory optimization	Safety stock, reorder point and EOQ from forecast demand, with a policy simulation.	Overstock/understock reduced 25-40%: shown on simulated stock only.
F-06	Interactive dashboard	Five Streamlit pages, filters, a promo-lift what-if slider and CSV downloads.	Real-time insights, exportable reports: CSV export done, PDF export not built.

9. Technology Stack

Category	Technology	Rationale / alternatives
Language	Python 3.11	Standard data science ecosystem.
Data processing	pandas, NumPy, scikit-learn	Cleaning, RFM features, K-Means, DBSCAN, evaluation.
Forecasting	Prophet, PyTorch Lightning (LSTM)	Prophet handles weekly and yearly seasonality; the LSTM was tried as a second model.
Churn model	XGBoost, SHAP, Optuna	Strong tabular model, explained with SHAP, tuned with Optuna (30 trials).
Data validation	Great Expectations 1.23	Checks for nulls and positive prices, quantities and revenue.
Experiment tracking	MLflow	Parameters, metrics and prediction files for every model run.
Drift monitoring	Evidently 0.7.23	Statistical drift tests per feature with HTML reports.
Dashboard	Streamlit, Streamlit Community Cloud	Fast to build, free public hosting.
Quality and CI	pytest, ruff, GitHub Actions	12 tests, linting, and a Docker build on every push.
Packaging	Docker (multi-stage), Kubernetes manifests	Image builds in CI; manifests written but not deployed.

Category	Technology	Rationale / alternatives
Orchestration	Airflow DAG file	Defines a weekly retraining schedule; Airflow was not run.

10. Output and Conclusion

Output:

- Cleaned dataset of 1,003,214 rows and 13 passing data-quality checks
- Six customer segments; Champions are 8.1% of customers and 54.4% of revenue
- Tuned Prophet forecast: MAPE 19.34%, WAPE 19.49% over 30 days, about 14% over 7 to 14 days
- Churn model: AUC 0.714 (random split), 0.768 (out-of-time); precision@top 20% of 0.686, about 1.8 times the base rate
- Reorder recommendations for the top 30 products, with a simulation showing 46% to 78% less unmet demand
- Drift detection that stays quiet on real data and alerts on simulated drift
- A public dashboard with five pages, passing CI, and tracked experiments

Conclusion:

RetailPulse takes raw retail transactions through cleaning, validation and feature engineering to four models, and presents the results in a public dashboard with monitoring and tests. The best forecast is a tuned Prophet model with a Saturday closure rule, and the churn model separates risky customers from safe ones better than simple baselines. The main findings are that a few customers carry most of the revenue and that nearly half of the customers are at risk or lost. Forecast and churn accuracy are below the targets in the brief, and the inventory results rest on simulated stock, so the next steps are more data and features, real stock and promotion data, and running the Airflow and Kubernetes setup.